

RecLib: a blazingly fast, principled recommendation systems library written in Rust

Miguel Martínez

Pontificia Universidad Católica de Chile

Santiago, Chile

martinez.m@uc.cl

Abstract

Recommendation systems demand numerical precision, tight control over memory and allocation, and correctness under concurrency, requirements that Rust, with its compile-time memory and thread-safety guarantees, and its zero-cost abstractions, is well suited to meet. Yet, the Rust ecosystem lacks a general, evaluation-oriented recommendation library that exploits these properties. This work presents RecLib, a session-based recommendation library with a safe-by-construction Rust core in which unsafe code is forbidden outright, an integrated ranking-metric and evaluation harness, and Python bindings via PyO3 that expose the compiled core without sacrificing its guarantees. RecLib’s initial model family comprises popularity and random baselines and three session-based nearest-neighbour recommenders (SKNN, V-SKNN, and S-SKNN). The library is validated along two axes against established references. For correctness, we show that RecLib reproduces the canonical session-rec implementations of Ludewig and Jannach to within floating-point tolerance, identical metrics and near-identical rankings, using session-rec as a correctness oracle. For speed, RecLib outperforms session-rec by three to four orders of magnitude at equal algorithm, and is competitive in per-prediction latency with Serenade, a production Rust recommender, while remaining ergonomically available from Python. RecLib thus offers a principled, safe, and rigorously validated foundation for recommendation research and deployment in Rust.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Software and its engineering** → *Software libraries and repositories*.

Keywords

Rust, sequential recommendation, next-item prediction, systems performance

ACM Reference Format:

Miguel Martínez. 2026. RecLib: a blazingly fast, principled recommendation systems library written in Rust. In *Proceedings of Recommender Systems (IIC3633)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IIC3633, Santiago, Chile

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXX>

1 Introduction

Rust is a modern systems programming language that offers low-level control over memory and execution while retaining a level of ergonomics that has historically been the domain of higher-level languages [14]. It pairs a mature toolchain: an integrated build system and package manager (cargo), a first-class test and documentation infrastructure, and a uniform dependency ecosystem (crates.io), with a compiler that rejects large classes of defects before a program ever runs. In this respect, Rust substantially improves on the developer experience of C and C++, whose power comes bundled with well-documented hazards around undefined behavior, manual memory management, and data races. Rust closes much of that gap without resorting to a garbage collector: its ownership and borrowing discipline is enforced statically, so memory safety and freedom from data races are guaranteed at compile time rather than paid for at runtime [4, 14].

These properties make Rust an unusually good fit for the constraints that recommendation systems impose. Such systems must be numerically precise, since small discrepancies in scoring propagate into observably different rankings; they must exercise strict control over memory layout and allocation, because throughput and latency depend on it; and they must remain correct under concurrency, because the workloads are naturally parallel over users, sessions, and candidate items. Rust addresses each of these directly. Its ownership model rules out use-after-free, double-free, and data races by construction [4], enabling what the community calls “fearless concurrency”, parallelism that the type system certifies as data-race-free.

Despite this fit, the Rust ecosystem lacks a recommendation systems library that does the language justice. The libraries that do exist (surveyed in Section 2) tend to be narrow in scope, tied to a single family of algorithms, or oriented toward a specific production deployment rather than toward general, reusable, and rigorously evaluated recommendation research. This gap is the opportunity from which the present work, RecLib, was born: not a port of an existing tool, but a library designed from the outset to be a principled home for recommendation algorithms.

The choice of programming language is therefore neither capricious nor arbitrary; it follows from a commitment to using appropriate tools for the task. Concretely, the library ships with integrated ranking-based evaluation metrics and an evaluation harness alongside the models themselves, so that correctness and quality can be measured within the same toolchain that produces predictions; it implements algorithmically precise models rather than approximations of convenience; maintains high test coverage; forbids unsafe code outright, so the memory-safety guarantees of the language hold across the entire core; it keeps its dependency surface as small

as practicable; and is accompanied by documentation kept current with the code. It is intended to integrate standard datasets directly into the library in future work, so that reproducible experiments are easier to setup.

The library implementation deliberately begin with sequential (session-based) recommendation models. The rationale is empirical rather than incidental: the canonical comparative studies of Ludewig and Jannach show that conceptually simple session-based methods, and nearest-neighbor approaches in particular, are remarkably powerful, frequently matching or outperforming far more complex neural models across a range of datasets and metrics [8, 9]. These methods thus offer an excellent accuracy-to-complexity ratio and serve as indispensable baselines for the field. They are also relatively simple to implement correctly, which makes them an ideal foundation on which to establish the evaluation harness, correctness guarantees, and engineering conventions before more elaborate model families are added.

2 Related Work

A small number of recommendation-related libraries have been written in Rust, but they differ from RecLib in scope, algorithmic focus, and intended use. The most relevant ones are reviewed here.

Serenade [1, 7] is the closest existing peer, and the only one squarely targeting session-based recommendation. It is a kNN-based recommender written in Rust and developed at bol.com, where it serves product recommendations in production. Its core contribution is VMIS-kNN, an index-based variant of a state-of-the-art nearest-neighbor algorithm for session-based recommendation that uses a prebuilt index to answer next-item queries with very low latency even when searching through hundreds of millions of interactions [7]. Serenade is best understood as a production recommender *system*, a deployable service centered on a single scalable algorithm, with hyperparameter tuning and a configuration-file interface, rather than as a general-purpose, extensible *library* of models with an integrated evaluation harness. Related to this work, it serves as a valuable Rust production peer and a point of comparison for runtime behavior.

Fwumious Wabbit [15] is a fast online machine-learning toolkit written in Rust and used at Outbrain, partially compatible with Vowpal Wabbit. It implements logistic regression and deep field-aware factorization machines, and its design emphasizes raw training and serving throughput via the hashing trick, a tight input-cache encoding, and heavy code specialization. Its factorization-machine models are applicable to click-through-rate prediction and related recommendation tasks. However, it is a general online-learning tool focused on a narrow set of linear and factorization models optimized for speed, not a recommendation library organized around recommendation-specific abstractions, and it does not address session-based next-item prediction.

Disco [5] provides recommendations for Rust using collaborative filtering. It supports user-based and item-based recommendations over both explicit and implicit feedback, built on high-performance matrix factorization: stochastic gradient descent with twin learners for explicit feedback and a conjugate-gradient method for implicit feedback. Disco is a clean, convenient collaborative-filtering library,

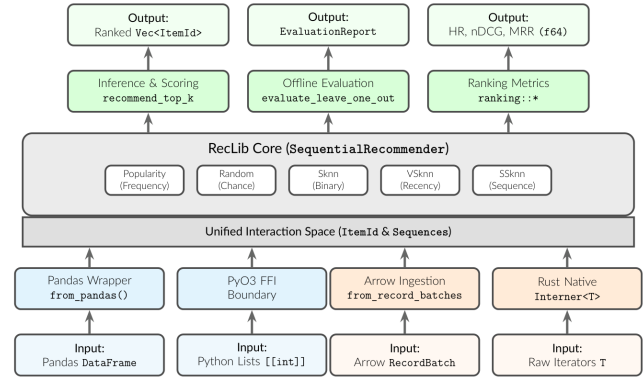


Figure 1: RecLib architecture. Heterogeneous inputs (bottom) are mapped onto a unified interaction space of interned item identifiers and sequences, consumed by the SequentialRecommender core, and served to the inference, offline-evaluation, and ranking-metric layers (top). The two left-hand ingestion paths cross the PyO3 FFI boundary; the two right-hand paths are native Rust.

but its scope is exactly that, classical matrix-factorization collaborative filtering, and it does not target sequential or session-based recommendation, nor does it aim to be a broad, evaluation-oriented platform spanning multiple model families.

Beyond these, the related crate **libmf** [6], also by the author of Disco, exposes large-scale sparse matrix factorization and underpins higher-level collaborative-filtering use cases; it is a factorization primitive rather than a recommendation library in its own right. Taken together, the existing Rust landscape covers production session-based serving (Serenade), fast online factorization models (Fwumious Wabbit), and collaborative-filtering matrix factorization (Disco, libmf), but leaves open the space RecLib is designed to fill: a principled, safe-by-construction, well-tested, and evaluation-first library that treats a family of recommendation algorithms as first-class, reusable components for both research and applications.

3 RecLib

This section delves into the internals of the library: how it is organized, what abstractions it is built on, the evaluation metrics and models it implements, and how it is released. RecLib is developed as an open project; its source lives in the GitLab repository [10], its narrative documentation is published as a Zensical site [13], and its Rust API reference is available on docs.rs [11]. Readers who prefer to follow along interactively can consult the cookbook [12], an executable notebook.

3.1 Architecture

RecLib is organized as a layered stack whose lowest layers accept data from several sources, whose middle layer unifies that data into a single internal representation, and whose upper layers perform inference, evaluation, and metric computation. Figure 1 shows this arrangement.

The interner. Recommendation data arrives as identifiers of arbitrary types, integer product codes, strings, or opaque keys, and the same identifier recurs across millions of interactions. Carrying these values around directly would be wasteful and makes the hot path depend on the size and hashing cost of the original type. To avoid this, RecLib funnels every input through an `Interner<T>`, which assigns each distinct raw identifier a small, dense, contiguous integer handle (an `ItemId`) the first time it is seen and returns the same handle on every subsequent occurrence. Interning gives the library three properties at once: a uniform internal item type regardless of the caller’s identifier type, so the core algorithms are written once against `ItemId`; constant-time equality and hashing on the hot path, since handles are machine integers; and a compact index space that lets models use dense arrays and bitsets rather than general hash maps. The interner is the pivot on which the “Unified Interaction Space” in Figure 1 rests: whichever ingestion path the data takes, it emerges on the other side as interned identifiers.

The sequence abstraction. Above the interner, RecLib represents user history not as isolated ratings but as *sequences* of interned items. A session is an ordered list of `ItemIds`, and the training corpus is a collection of such sequences. This is the natural data model for session-based recommendation: the models in Section 3.3 consume a prefix of the current session and past sessions, and order carries signal (an item interacted with more recently is treated differently from one seen at the start of the session). By committing to sequences as the assumed abstraction, the library keeps a single input contract across every model, where each recommender is a function from a current-session prefix and a body of past sequences to a ranked list of items, which is what makes models interchangeable within the same harness.

The evaluation harness. RecLib ships evaluation as a first-class part of the library rather than as an external script. The harness implements a leave-one-out protocol: for each test sequence, the final interaction is held out as the ground-truth target and the preceding prefix is presented to the model, which produces a ranked list of candidates; the position of the held-out item in that list drives the metrics of Section 3.2. Aggregated over the test sequences, this yields an `EvaluationReport` (the middle output branch of Figure 1) that summarizes accuracy without the caller having to assemble a bespoke evaluation loop. Because the harness consumes the same sequence abstraction that the models do, any model that satisfies the common contract can be evaluated without modification.

The FFI boundary. RecLib is a Rust library first, but it is designed to be usable from Python without sacrificing the guarantees of its core. The Python bindings are built with PyO3, which provides Rust bindings for the CPython interpreter and the machinery for exposing Rust types and functions as native Python objects [16]. Concretely, PyO3 lets RecLib present a Python-facing surface (the `from_pandas` wrapper and the PyO3 FFI ingestion path on the left of Figure 1) while the heavy computation remains in compiled, `#![forbid(unsafe_code)]` Rust. Data supplied from Python, like lists of lists, or Arrow record batches ingested through `from_record_batches`, cross this boundary once, is interned, and thereafter is processed entirely by the native core; results cross back as ordinary Python values. The two native paths on the right

(Arrow ingestion and the raw-iterator API over the `Interner<T>`) let Rust callers skip the FFI entirely. The design thus serves both audiences from a single implementation: the algorithmic code, the metrics, and the harness exist once, and PyO3 makes them available to Python at the edge.

3.2 Metrics

RecLib implements the standard top- k ranking metrics used in the session-based literature [8]. In every case, evaluation follows the leave-one-out protocol of Section 3.1: for a test sequence the model returns a ranked list, and the metric compares the rank of the single held-out target item against the cutoff k . Let N be the number of evaluated sequences, let rank_u denote the position (starting at 1) of the held-out target for sequence u in the model’s ranked output, and let $\mathbb{I}[\cdot]$ be the indicator function. All metrics are computed as $f64$ values.

Hit Rate (HR@ k). The hit rate, also called recall in the single-target setting, is the fraction of test sequences whose held-out item appears within the top k positions. It measures whether the correct item was retrieved at all, ignoring where in the list it landed.

$$\text{HR}@k = \frac{1}{N} \sum_{u=1}^N \mathbb{I}[\text{rank}_u \leq k]$$

Mean Reciprocal Rank (MRR@ k). The mean reciprocal rank rewards placing the held-out item near the top of the list. Each sequence contributes the reciprocal of the target’s rank when that rank is within the cutoff, and zero otherwise, so a correct item at position 1 contributes 1, at position 2 contributes $\frac{1}{2}$, and so on.

$$\text{MRR}@k = \frac{1}{N} \sum_{u=1}^N \frac{\mathbb{I}[\text{rank}_u \leq k]}{\text{rank}_u}$$

Normalized Discounted Cumulative Gain (nDCG@ k). The nDCG applies a logarithmic discount to the position of the held-out item. With a single relevant item per sequence the ideal DCG is 1, so the normalized gain reduces to the discounted gain of the target when it falls within the cutoff. This penalizes lower ranks more gently than MRR while still favoring higher placements.

$$\text{nDCG}@k = \frac{1}{N} \sum_{u=1}^N \frac{\mathbb{I}[\text{rank}_u \leq k]}{\log_2(\text{rank}_u + 1)}$$

3.3 Models

RecLib’s initial model family consists of two trivial baselines and three nearest-neighbor session-based recommenders. Each model implements the common contract of Section 3.1, mapping a current-session prefix and a body of past sequences to a score per candidate item, and differs only in how it computes those scores. Each is described in turn, giving for the neighbor-based models the session similarity function that drives scoring, and then work a single flagship example through all three.

Popularity (frequency baseline). The popularity model ignores the current session entirely and scores each candidate item by how often it occurs in the training corpus. Writing $f(i)$ for the number of training sequences containing item i , the score is simply

$$\text{score}_{\text{Pop}}(i) = f(i).$$

It serves as a lower bound that any session-aware model should beat.

Random (chance baseline). The random model assigns each candidate an independent uniform score, providing a chance-level reference:

$$\text{score}_{\text{Rand}}(i) \sim \mathcal{U}(0, 1).$$

Session-based kNN (SKNN). SKNN is the neighborhood scheme introduced by Jannach and Ludewig [3] and studied in depth by Ludewig and Jannach [8]. It treats each session as an unordered set of items and finds the past sessions most similar to the current one, then recommends the items those neighbors contain. Similarity between the current session s and a past session n is the cosine similarity of their binary item-set indicator vectors,

$$\text{sim}_{\text{SKNN}}(s, n) = \frac{|s \cap n|}{\sqrt{|s| |n|}},$$

and the score of a candidate item i is the sum of the similarities of the neighbor sessions that contain it, $\text{score}(i) = \sum_{n \in \mathcal{N}(s)} \text{sim}(s, n) \mathbb{1}[i \in n]$, where $\mathcal{N}(s)$ is the neighborhood of s . The “Binary” label in Figure 1 refers to this set-based treatment.

Vector-multiplication SKNN (V-SKNN). V-SKNN [8] refines SKNN by acknowledging that recent actions in the current session are more indicative of intent than earlier ones. Rather than treating the current session as a flat set, it weights each of its items by recency: for a current session of length m whose item at position p (with $p = 1$ oldest and $p = m$ most recent) we assign a linear weight $w(p) = p/m$. Similarity to a past session n is then the sum of the current-session weights over the shared items,

$$\text{sim}_{\text{VSKNN}}(s, n) = \sum_{i \in s \cap n} w(\text{pos}_s(i)),$$

where $\text{pos}_s(i)$ is the position of item i within the current session. Candidate scoring then proceeds as in SKNN, summing these recency-aware similarities over the neighbors containing each candidate. This is the “Recency” variant in Figure 1.

Sequential SKNN (S-SKNN). S-SKNN [8] injects order on the scoring side. It uses the same binary neighbor similarity as SKNN, but when a neighbor n votes for a candidate, its vote is scaled by how recently the current session’s items appear in n : neighbors that match the most recent part of the current session count for more. Concretely, defining the sequential weight of neighbor n as the recency weight $w(p) = p/m$ of the *most recent* current-session item that also occurs in n ,

$$\text{score}_{\text{SSKNN}}(i) = \sum_{n \in \mathcal{N}(s)} \text{sim}_{\text{SKNN}}(s, n) w\left(\max_{j \in s \cap n} \text{pos}_s(j)\right) \mathbb{1}[i \in n].$$

This is the “Sequence” variant in Figure 1.

A flagship worked example. Consider a training corpus of four past sessions over items $\{A, B, C, D, E\}$,

$$s_1 = \langle A, B, C, D \rangle, \quad s_2 = \langle A, B, D \rangle, \quad s_3 = \langle B, C, E \rangle, \quad s_4 = \langle A, C, D \rangle,$$

and a current session $s = \langle A, B \rangle$ for which we want the next-item recommendations. We take the full corpus as the neighborhood.

For **SKNN**, the binary cosine similarities are $\text{sim}(s, s_1) = 2/\sqrt{4 \cdot 2} \approx 0.707$, $\text{sim}(s, s_2) = 2/\sqrt{3 \cdot 2} \approx 0.816$, $\text{sim}(s, s_3) = 1/\sqrt{3 \cdot 2} \approx 0.408$, and $\text{sim}(s, s_4) = 1/\sqrt{3 \cdot 2} \approx 0.408$. Summing these over the sessions that contain each unseen candidate gives $\text{score}(C) = 0.707 + 0.408 + 0.408 \approx 1.524$, $\text{score}(D) = 0.707 + 0.816 + 0.408 \approx 1.932$, and $\text{score}(E) = 0.408$, so the ranking is $D \succ C \succ E$.

For **V-SKNN**, the current-session recency weights are $w(A) = 1/2$ and $w(B) = 1$. The session similarities become $\text{sim}(s, s_1) = w(A) + w(B) = 1.5$, $\text{sim}(s, s_2) = 1.5$, $\text{sim}(s, s_3) = w(B) = 1.0$ (as s_3 shares only B), and $\text{sim}(s, s_4) = w(A) = 0.5$ (as s_4 shares only A). Summing over containing sessions yields $\text{score}(C) = 1.5 + 1.0 + 0.5 = 3.0$, $\text{score}(D) = 1.5 + 1.5 + 0.5 = 3.5$, and $\text{score}(E) = 1.0$: the ranking is again $D \succ C \succ E$, but the recency weighting widens B ’s influence, so sessions sharing the most recent item B are pulled up.

For **S-SKNN**, the neighbor similarities are the binary SKNN values above, each scaled by the recency weight of the most recent current-session item present in the neighbor. Sessions s_1 , s_2 , and s_3 all contain B (the most recent item, weight 1), while s_4 contains only A (weight 1/2). The scaled votes give $\text{score}(C) = 0.707 \cdot 1 + 0.408 \cdot 1 + 0.408 \cdot 0.5 \approx 1.320$, $\text{score}(D) = 0.707 \cdot 1 + 0.816 \cdot 1 + 0.408 \cdot 0.5 \approx 1.728$, and $\text{score}(E) = 0.408 \cdot 1 \approx 0.408$, for the ranking $D \succ C \succ E$.

All three models agree on the top recommendation D and on the overall order $D \succ C \succ E$ for this small example, but they arrive there by weighting the evidence differently: SKNN counts shared items, V-SKNN emphasizes similarity to sessions that overlap on recent items, and S-SKNN discounts neighbors whose overlap with the current session is confined to older items. On realistic data these differences change not only the scores but the rankings themselves, which is precisely what the evaluation harness of Section 3.2 is built to measure.

3.4 Deployment Model

RecLib is distributed through three artifacts, each produced by a distinct release path from a single source of truth. Figure 2 depicts the arrangement: the GitLab repository `reclib/reclib` at the bottom feeds three upward paths that terminate in the documentation site, the Python package, and the Rust crate.

The first two paths are fully automated through GitLab CI/CD. A commit to the main branch triggers the pages job, which builds the Zensical documentation and publishes it to GitLab Pages as the project’s documentation site. Independently, pushing a version tag matching `v*` triggers the build and publish jobs, which use `maturin` to compile the PyO3 bindings into Python wheels and a source distribution and upload them to PyPI. Tying the wheel-building path to version tags rather than to ordinary commits keeps published releases aligned with explicit, tagged versions.

The third path, publishing the Rust crate source to `crates.io`, is deliberately kept as a manual step. A developer runs `cargo publish -p reclib` from a local terminal to release the core crate. Separating this from the automated tag pipeline reflects that the crate registry and the Python index have different release cadences and review

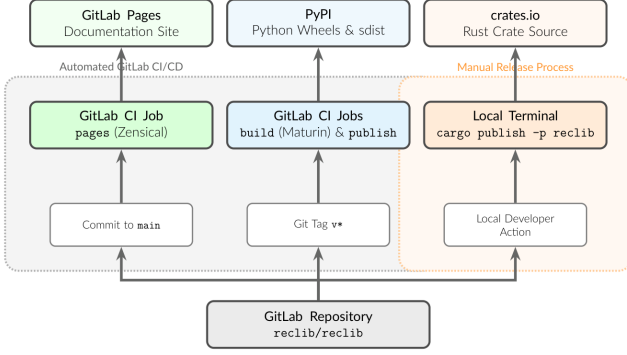


Figure 2: The RecLib deployment model. From the single GitLab repository, two automated GitLab CI/CD paths publish the documentation site and the Python wheels, while a third, manual path publishes the Rust crate source to crates.io.

expectations, and it gives the maintainer an explicit checkpoint before the Rust source is published. Together, the three paths let a single tagged state of the repository fan out into documentation, a Python package, and a Rust crate, each reaching its own audience through the appropriate channel.

4 Benchmark

RecLib is evaluated along two axes: *algorithmic correctness* and *run-time speed*. A third axis, memory footprint, is of clear interest for a systems-oriented library but fell outside the time constraints of this work; we return to it as future work in Section 5. All experiments use the MovieLens 100K dataset [2], treating each user’s time-ordered history as a session. A single canonical preprocessing step is shared across every library, so that no observed difference can be attributed to differences in data preparation, and each library is presented with the identical set of (prefix, target) test instances. The entire benchmark is containerized and reproducible from a single command. We compare against two references that implement the same session-based nearest-neighbour family: **session-rec** [8], the framework of Ludewig and Jannach, and **Serenade** [7], the production Rust recommender discussed in Section 2.

4.1 Correctness

The central correctness question is whether RecLib computes the *same function* as an established, independently implemented reference. We answer it against **session-rec** [8, 9], the framework accompanying the comparative studies of Ludewig and Jannach. **session-rec** is the natural oracle for this task: it is the reference implementation against which the session-based results in the literature are reported, its authors formalized several of the very variants RecLib implements, and it has been scrutinized and reused by the community for years. Agreement with **session-rec** is therefore agreement with the field’s operational definition of these models.

Exact bit-level agreement is neither achievable nor the right target: RecLib computes in f32 and **session-rec** in f64, so floating-point ties resolve differently. We instead validate equivalence in

Table 1: Ranking accuracy on MovieLens 100K ($n = 750$ test instances, cutoff @20). RecLib and **session-rec are configured to the same algorithm; SKNN and V-SKNN agree within floating-point tolerance. **session-rec** provides no native sequential-SKNN model, so S-SKNN is reported for RecLib only. Popularity is a non-personalized baseline.**

Model	Library	HR@20	MRR@20	nDCG@20
SKNN	RecLib	0.0947	0.0149	0.0316
SKNN	session-rec	0.0947	0.0149	0.0316
V-SKNN	RecLib	0.1187	0.0215	0.0419
V-SKNN	session-rec	0.1173	0.0217	0.0418
S-SKNN	RecLib	0.1480	0.0330	0.0578
Popularity	RecLib	0.0840	0.0160	0.0301

layers. First, we match the configuration so that both libraries compute the same underlying function; notably, this requires aligning the scoring *formulas* rather than parameter names, since **session-rec** exposes its weighting and similarity options under labels that do not map directly to their mathematical definitions. Second, with neighbour sampling disabled for determinism and the same shared instances presented to both, we measure *ranking parity* (do the top- k lists agree, item by item?) and *metric parity* (do HR, MRR, and nDCG agree, within tolerance?).

Tables 1 and 2 report the results on $n = 750$ test instances. For **SKNN**, the agreement is essentially exact: the two libraries produce identical metrics ($\max |\Delta| = 0$), agree on the top-ranked item for 99.2% of instances, and reproduce the entire ordered top-20 for 98.5%. For **V-SKNN**, once the configuration is reconciled, the metrics coincide within tolerance ($\max |\Delta| = 1.4 \times 10^{-3}$), the top item agrees for 99.1% of instances, and the top-20 sets are near-identical (mean Jaccard 0.995). The stricter exact-order measure is lower, at 86.4%, but this residual is attributable to tie-breaking rather than to any algorithmic difference: V-SKNN’s recency weighting produces many equal scores, and the two libraries order such ties by different conventions (RecLib by interned first-appearance order, **session-rec** by raw identifier order). That the aggregate metrics nonetheless agree to within 1.4×10^{-3} confirms the underlying computation is equivalent. The @10 cutoff, omitted for space, exhibits the same pattern.

Two models fall outside this parity analysis, for different reasons. **S-SKNN** has no counterpart in the **session-rec** branch we benchmark against, because its nearest-neighbour module ships no sequential-SKNN class, so we report S-SKNN’s metrics for RecLib alone (Table 1) and cannot cross-validate it here; supplying it with an oracle is left to future work. **Serenade** is excluded from the correctness comparison altogether: its evaluator uses an incompatible evaluation protocol (multi-item relevance, a different cutoff, and its own internally generated prediction points), which renders its metrics numerically incomparable with the single-target protocol used here. **Serenade** participates only in the speed comparison of Section 4.2.

Table 2: Ranking parity between RecLib and session-rec on the shared instances ($n = 750$, cutoff @20, neighbour sampling disabled). *Top-1* is the fraction of instances whose top-ranked item agrees; *Exact @20* the fraction whose entire ordered top-20 agrees; *Jaccard* the mean set overlap of the two top-20 lists; $\max |\Delta|$ the largest difference across HR, MRR, and NDCG.

Model	Top-1 (%)	Exact @20 (%)	Jaccard	$\max \Delta $
SKNN	99.2	98.5	0.9995	0.0000
V-SKNN	99.1	86.4	0.9951	0.0014

4.2 Speed

We measure both training (*fit*) and inference. Inference is reported as throughput (prefixes scored per second) and as per-query latency percentiles. RecLib is measured *twice*: as a native Rust binary, giving the performance ceiling with no language-boundary cost, and through its PyO3 bindings, giving the figure a Python caller actually observes. The Python baseline is session-rec; the Rust production peer is Serenade. All measurements use the same shared instances, a single thread, neighbour sampling disabled, and report the median of five repetitions.

Table 3 reports V-SKNN, the one model shared by all three libraries. Native RecLib sustains roughly 6,990 predictions per second against session-rec’s 0.72, a speedup of about 9,700 $\times$; through PyO3 the figure is about 1,580 $\times$, the gap between the two (a factor of roughly six) being the cost of crossing the FFI boundary on every call. The latency distributions tell the same story and expose session-rec’s pathological tail: its median per-query latency is 840 ms and its 95th percentile 4.7 s, driven by long session prefixes re-scanned in interpreted Python, whereas native RecLib stays flat at 0.13 ms median and 0.28 ms at the 95th percentile. Serenade attains a competitive 0.44 ms per prediction, between the two RecLib figures; but MovieLens 100K, with this number of sessions, is far too small for its index to pay off, so this should be read as a reference point rather than a demonstration of its scale advantage, which is designed for corpora of hundreds of millions of interactions. The speedups are consistent across the family: SKNN and S-SKNN show the same three-to-four orders of magnitude over session-rec.

One caveat frames these numbers. They are obtained with neighbour sampling disabled, as it is the configuration required for exact-parity correctness, so both libraries perform the same exhaustive work. The reported speedup therefore reflects implementation efficiency on identical computation, not production throughput: with sampling enabled, as in deployment, session-rec would be considerably faster and the gap would narrow. The comparison is honest precisely because it holds the algorithm fixed.

5 Conclusions

We have presented RecLib, a session-based recommendation library that treats the properties of Rust not as an implementation detail but as the foundation of a *principled* design. Its core forbids unsafe code outright, so the language’s memory and thread-safety guarantees hold across the entire library; it ships ranking metrics and a leave-one-out evaluation harness as first-class components, so that quality

Table 3: V-SKNN runtime on MovieLens 100K ($n = 200$ test instances, median of five repetitions, single thread, neighbour sampling disabled). Throughput is inference queries per second; speedup is throughput relative to session-rec. RecLib is measured natively and through its PyO3 bindings. [†]Serenade reports a single per-prediction latency from its evaluator; on a corpus this small it cannot exercise the index that gives it its scale advantage.

Library	Fit (ms)	Thpt. (q/s)	p95 (ms)	Speedup
RecLib (Rust)	1.2	6989	0.28	9671 $\times$
RecLib (PyO3)	3.3	1142	1.21	1581 $\times$
session-rec	62.7	0.72	4734	1.0 $\times$
Serenade [†]	<i>n/a</i>	<i>n/a</i>	0.44	—

is measured within the same toolchain that produces predictions; and it exposes this compiled core to Python through PyO3 without duplicating a single line of algorithmic logic.

The value of this discipline is what the benchmark makes concrete. Because RecLib is evaluation-first, its correctness is *demonstrated* rather than asserted: it reproduces the canonical session-rec implementations to within floating-point tolerance on SKNN and V-SKNN. Because RecLib is built on a safe, allocation-conscious Rust core, it is also fast: three to four orders of magnitude faster than the reference framework at equal algorithm, and competitive in per-prediction latency with a production Rust recommender, all while remaining ergonomically available to the Python research community.

This work is a foundation rather than a finished system. The immediate next steps are to broaden the model family beyond the nearest-neighbour baselines, like factorized personalized Markov chains (FPMC) and neural sequential models such as GRU4Rec and SASRec, and to extend the benchmark itself: to characterize the memory footprint that a systems-oriented library ought to report, to integrate standard session datasets for reproducible large-scale evaluation, and to supply an oracle for the models, such as S-SKNN, that currently lack one. What the present work establishes is the groundwork on which those additions can rest: a correct, fast, safe-by-construction, and rigorously validated core that makes RecLib a principled home for recommendation algorithms in Rust.

References

- [1] bol.com. 2022. Serenade: Session-Based Recommender System. <https://github.com/bolcom/serenade>. Accessed: 2026-07-07.
- [2] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19. doi:10.1145/2827872
- [3] Dietmar Jannach and Malte Ludewig. 2017. When Recurrent Neural Networks Meet the Neighborhood for Session-Based Recommendation. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys)*. Association for Computing Machinery, 306–310. doi:10.1145/3109859.3109872
- [4] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the Foundations of the Rust Programming Language. In *Proceedings of the ACM on Programming Languages (POPL)*, Vol. 2. Association for Computing Machinery, Article 66. doi:10.1145/3158154
- [5] Andrew Kane. 2021. Disco: Recommendations for Rust Using Collaborative Filtering. <https://github.com/ankane/disco-rust>. Accessed: 2026-07-07.
- [6] Andrew Kane. 2021. libmf: Large-Scale Sparse Matrix Factorization for Rust. <https://github.com/ankane/libmf-rust>. Accessed: 2026-07-07.

- [7] Barrie Kersbergen, Olivier Sprangers, and Sebastian Schelter. 2022. Serenade – Low-Latency Session-Based Recommendation in e-Commerce at Scale. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*. Association for Computing Machinery, 150–159. doi:10.1145/3514221.3517901
- [8] Malte Ludewig and Dietmar Jannach. 2018. Evaluation of Session-Based Recommendation Algorithms. *User Modeling and User-Adapted Interaction* 28, 4–5 (2018), 331–390. doi:10.1007/s11257-018-9209-6
- [9] Malte Ludewig, Noemi Mauro, Sara Latifi, and Dietmar Jannach. 2021. Empirical Analysis of Session-Based Recommendation Algorithms: A Comparison of Neural and Non-Neural Approaches. *User Modeling and User-Adapted Interaction* 31, 1 (2021), 149–181. doi:10.1007/s11257-020-09277-1
- [10] Miguel Martínez. 2026. RecLib: A Principled Sequential Recommendation Library in Rust. <https://gitlab.com/reclib/reclib>. Accessed: 2026-07-07.
- [11] Miguel Martínez. 2026. RecLib API Reference (docs.rs). <https://docs.rs/reclib/0.1.0-alpha.2/reclib/index.html>. Accessed: 2026-07-07.
- [12] Miguel Martínez. 2026. RecLib Cookbook. https://molab.marimo.io/notebooks/nb_Kjj396z9vzDGz7TsFZ3nvH. Accessed: 2026-07-07.
- [13] Miguel Martínez. 2026. RecLib Documentation. <https://reclib.gitlab.io/reclib/>. Accessed: 2026-07-07.
- [14] Nicholas D. Matsakis and Felix S. Klock, II. 2014. The Rust Language. *ACM SIGAda Ada Letters* 34, 3 (2014), 103–104. doi:10.1145/2692956.2663188
- [15] Outbrain, Inc. 2024. Fwumious Wabbit: A Fast On-Line Machine Learning Toolkit Written in Rust. https://github.com/outbrain-inc/fwumious_wabbit. Accessed: 2026-07-07.
- [16] The PyO3 Project and Contributors. 2024. PyO3: Rust Bindings for the Python Interpreter. <https://github.com/PyO3/pyo3>. Accessed: 2026-07-07.

A LLM usage statement

Large Language Models (LLMs) were employed for improving the writing clarity and language fluency of this paper.

Received 07 July 2026